Mini-Project 2: Control task in a Continuous Environment

Reinforcement Learning Course

Spring Semester 2026

Grading and submission

Grading: The mini-project is mandatory and counts towards 30% of the final grade. You will be graded on the results you obtained and your answers to the questions, and a fraction of the grading is also dedicated to the quality and clarity of the report.

Teams: You will have to work in teams of two students. To find a project partner if you don't have one yet, you can use the EdStem forum (https://edstem.org/eu/courses/2295/discussion).

Final report: Your submission should be the interactive notebook .ipynb. Submit the notebook with all the plots and answers, but running it should reproduce all your results and plots (if we run it after placing it in the same folder with the environment code that we provided). The code itself will not be graded, only your answers to the questions. Clearly label your answers to the questions and the plots that form part of the answer. Write concisely in complete English sentences and present figures carefully (axis labels, legends etc.). Write concisely: go straight to the point in answering the question.

Submission: You will have to submit your .ipynb notebook on the Moodle page. The file name should have the structure MPX_NameMember1_NameMember2.ipynb, where X is the miniproject number (either 1 or 2). If you work in teams both of you should submit the same file. On Moodle, make sure you press the submit button before the deadline.

Deadline: The deadlines to submit notebook are:

- May 26th 2026, at 11.55 pm (fraud-detection slots: May 27th and 28th, 2026)
- June 2nd 2026, at 11.55 pm (fraud-detection slots: June 3rd and 4th, 2026)

The early deadline allows you to do the fraud detection before the exam preparation period, which you might find convenient.

Regulations and fraud detection: After the submission, we will perform a fraud detection interview which consists of a few questions about your report and your code. You will need to attend in-person (both members of the team). Choose your deadline accordingly.
The mini-project is graded and the same rules as for exams apply ("Ordonnance sur le contrôle des études à l'EPFL"). In short: EPFL takes any form of cheating and plagiarism very serious. The mini-project has to be your own work. You are not allowed to share code or answers across teams. However you can discuss ideas between teams and ask questions on the Ed forum. If you split the work between the two team members, both team members have to understand all parts of the mini-project. Note: The goal of this meeting is fraud detection; it is neither an exam nor a presentation.

1 Introduction

In this project, we will consider an environment with a continuous state space. The project is organized around three families of methods:

1. **TD learning with function approximation**: Q-learning applied to a discretized or radial-basis-function representation of the state space.
2. **Policy gradient**: directly optimizing a parameterized policy using the REINFORCE algorithm.
3. **Actor-critic**: combining a learned policy (actor) with a learned value function (critic).

Environment. We use an adapted version of the Acrobot Gymnasium environment. The Acrobot is a two-link pendulum (see Figure 1). The agent controls a torque applied at the joint between the two links, with the goal of swinging the free end of the lower link above a target height threshold.

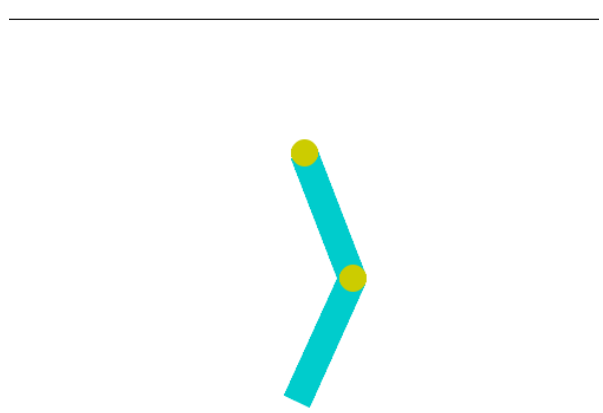

Figure 1: The Acrobot system. The agent applies torques at the joint between the two links to swing the lower link above the target line.

The episode dynamics are as follows:

- At each step, the agent receives a reward of -1, except when the free end of the lower link reaches or exceeds the target height, at which point the episode ends with a reward of 0.
- Episodes are truncated after 500 steps if the goal is not reached. The total reward therefore ranges from -500 (never succeeded) to -1 (succeeded on the first step, which is practically impossible from the initial state).
- The agent's goal is to reach the target height as quickly as possible, i.e. to maximize the (negative) reward, or equivalently to minimize the number of steps before success.

The state is a six-dimensional continuous vector:

$$s = (\cos\theta_1,\ \sin\theta_1,\ \cos\theta_2,\ \sin\theta_2,\ \dot{\theta}_1,\ \dot{\theta}_2),$$

where θ_1 and θ_2 are the angles of the two links, and $\dot{\theta}_1$, $\dot{\theta}_2$ are their angular velocities.

There are three discrete actions: apply a negative torque (-1), apply no torque (0), or apply a positive torque ($+1$) at the joint. For a refresher, a torque in this setting acts as an acceleration for the angle.

Provided code. The environment has the following interface:

- `reset()`: resets the environment to the initial state and returns it.
- `available()`: returns the list of available actions: `[0, 1, 2]` (always).
- `n_actions`: total number of actions: 3.
- `state_space`: a `(low, high)` tuple of lower and upper bounds for each state dimension.
- `get_state()`: returns the current state as described above.
- `do_action(action)`: executes the given action; returns `(next_state, reward)`.
- `reward()`: returns the reward from the most recent step.
- `save_gif(filename)`: saves the current episode (since the last reset) as a GIF.

The notebook skeleton also provides:

- An abstract base class `AbstractAgent` with the interface `observe(state, action, reward, next_state, done)` and `act(state)`. The function `observe` will be called after each environment transition, and should update the parameters of the agent. The function `act` should output an action following the agent's policy.
- A function `run_agent(env, agent, num_episodes)` (to be completed) that runs the agent and returns `rewards` and `episode_length` lists.
- A function `gif_episode(env, agent, filename)` that simulates one episode (without calling `observe`) and saves it as a GIF.

You can implement additional functions in the notebook, **but do not modify the environment file**.

2 TD Learning

2.1 Random Agent

Implement the `RandomAgent` class, which selects a uniformly random action at every step. Use `gif_episode` to visualize one episode of the random agent.

Complete the `run_agent` function described above. Also, implement the following two diagnostic functions:

- `plot_rewards(rewards, window_size)`: plots the rolling-average episode reward over time (averaged over a window of `window` episodes).
- `plot_success(rewards, window_size)`: plots the rolling fraction of episodes in which the goal was reached.

Use both functions to evaluate the random agent over a sufficient number of episodes.

Questions

a. How often does the random agent reach the target height?

b. What is the average reward collected by the random agent? Interpret this value in terms of episode duration.

2.2 Q-Learning with State Discretization

Since Q-learning requires a finite, enumerable state space, we must first map the continuous state to a discrete representation. A simple approach is to partition each state dimension into a fixed number of equally spaced bins.

1. Implement the `StateDiscretizer` class. At initialization it receives the environment (to read `state_space`) and an integer `n_bins` specifying the number of bins per dimension. When called with a continuous state, it returns a discretized version of it. Test your implementation with the provided test code.

2. Implement a `QLearningAgent` that uses `StateDiscretizer` internally and follows the standard update rule:
$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big].$$
Break Q-value ties randomly and use an ε-greedy policy with $\varepsilon = 0.1$.

3. Run the agent for several values of `n_bins`. Plot the performance of each configuration and of the random agent on the same plot.

Questions

a. For a given number of bins per dimension n, how many discrete states are there in total? How does this scale with the state dimension d?

b. What are the trade-offs of using a small number of bins? What are the trade-offs of using a large number of bins?

c. On average, which agent (random or Q-learning) will first reach the target height? Would this change with a different value of ε or a different Q-value initialization? Justify.

d. Does the Q-learning agent improve its performance over time? Does it manage to reach the goal in every episode eventually? Why or why not?

e. How does performance depend on `n_bins`? Which value gives the best results on this environment?

2.3 Q-Learning with Radial Basis Functions

Bin-based discretization is coarse: all states that fall in the same bin are treated as identical, regardless of how close or far they are within the bin. Radial basis functions (RBFs) offer a smoother alternative. Instead of assigning a state to a single bin, an RBF representation produces a vector of soft activations, one per basis center, that vary continuously with the state.

RBF features. Given K centers $\{c_j\}_{j=1}^{K}$ where $c_j \in \mathbb{R}^d$ and per-dimension length scales $\{\sigma_d\}_{d=1}^{D}$, the j-th RBF feature of state $s \in \mathbb{R}^d$ is:

$$\phi_j(s) = \exp\left(-\sum_{d=1}^{D} \frac{(s_d - c_{j,d})^2}{2\sigma_d^2}\right).$$

The full feature vector is $\phi(s) = [\phi_1(s), \ldots, \phi_K(s)]^\top \in \mathbb{R}^K$.

Linear Q-function. For each action a, we maintain a weight vector $w_a \in \mathbb{R}^K$. The Q-value is then:

$$Q(s, a \mid w) = w_a^\top \phi(s) = \sum_{j=1}^{K} w_{a,j}\, \phi_j(s).$$

Semi-gradient update. The TD error for a transition (s, a, r, s') is:

$$\delta = r + \gamma \max_{a'} Q(s', a' \mid w) - Q(s, a \mid w).$$

We minimize the squared TD error $\mathcal{L}(w) = \frac{1}{2}\delta^2$ by computing the gradient with respect to the weights w while treating the target $r + \gamma \max_{a'} Q(s', a' \mid w)$ as constant (hence "semi-gradient", as taught in class). The update rule is:

$$w_{a,j} \leftarrow w_{a,j} - \alpha \frac{\partial}{\partial w_{a,j}} \mathcal{L}(w)$$

1. Implement `rbf_features(state, centers, sigmas)`, which returns the feature vector $\phi(s)$ given an array of centers and a vector of per-dimension length scales.
2. Implement the `RBFAgent` class. At initialization it receives `n_centers_per_dim` and constructs a regular grid of centers that covers the state space. Set each σ_d to half the spacing between adjacent centers along dimension d. The `observe` method applies the semi-gradient update above.
3. Run the agent and plot its performance for several combinations of `n_centers_per_dim` and learning rate α.

Questions

a. Derive the partial derivative of $\mathcal{L}(w) = \frac{1}{2}\delta^2$ with respect to $w_{a,j}$, treating the target as a constant. Show your calculations and state the resulting update rule.

b. After observing a transition (s_t, a_t, r_t, s_{t+1}), which of the weights $w_{a,j}$ get updated and which stay unchanged?

c. What would differ between the bin-based Q-learning agent and the RBF agent in their treatment of a previously unseen state?

d. Does the RBF agent improve over time? Does it reach the goal consistently? Why or why not?

e. How does performance depend on `n_centers_per_dim` and α? Which combination yields the best results?

3 Policy Gradient (week 6)

TD-learning methods learn a value function and derive a policy from it. Policy gradient methods take a different approach: they directly parameterize the policy $\pi_\theta(a \mid s)$ and optimize the parameters θ by gradient ascent on the expected return.

3.1 REINFORCE with a Linear Softmax Policy

Policy parameterization. We represent the policy as a softmax over a linear function:

$$\pi_\theta(a \mid s) = \frac{\exp\big(\theta_a^\top \phi(s)\big)}{\sum_{a'} \exp\big(\theta_{a'}^\top \phi(s)\big)},$$

where $\phi(s) \in \mathbb{R}^K$ is the same RBF feature vector as before, and $\theta_a \in \mathbb{R}^K$ is a weight vector for action a.

REINFORCE is a Monte Carlo policy gradient algorithm. It runs a full episode, collects the trajectory $\{(s_t, a_t, r_t)\}_{t=0}^T$, and then updates the policy parameters using the log-derivative trick. The pseudocode of the algorithm is given in Figure 2 below

1. Derive the expression for $\nabla \ln \pi_\theta(a_t \mid s_t, \theta)$.

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

From book:
Sutton and Barto, 2018

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, \boldsymbol{r_1} \ldots, S_{T-1}, A_{T-1}, \boldsymbol{r_T}$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} \boldsymbol{r_k}$ $\qquad (G_t)$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

Figure 2: REINFORCE algorithm

2. Implement the `REINFORCEAgent` class with an RBF feature representation and the softmax policy. Note that REINFORCE requires a complete episode before updating; your `observe` method should store transitions, and perform all the updates once the episode is over.

3. Run the agent and plot its performance over multiple runs. Try different values for the learning rate and the number of centers per dimension.

Questions

a. What gradient expression did you derive? Show your calculations.

b. What are the main differences between REINFORCE and Q-learning with RBFs in terms of (i) what they learn, (ii) how they update, and (iii) the kind of policy they produce?

c. Could we use an ε-greedy policy with REINFORCE? Why or why not?

d. Does the agent improve over time? Comment on your plots.

3.2 REINFORCE with a Multi-Layer Perceptron

In this section, the policy is an MLP that maps the raw state $s \in \mathbb{R}^6$ directly to action logits (no manual feature engineering needed):

$$\text{logits}(s) = W_2\, \sigma(W_1 s + b_1) + b_2, \qquad \pi_\theta(a \mid s) = \text{softmax}(\text{logits}(s))_a,$$

where σ is the ReLU activation function and $\theta = \{W_1, b_1, W_2, b_2\}$.

Because computing the gradient of the MLP by hand is tedious, we will use automatic differentiation via PyTorch. Most of the PyTorch code is provided. You need to fill in the `act` and `observe` methods using the provided helper functions.

1. Complete the `REINFORCEAgentMLP` class by implementing the `act` and `observe` methods. The REINFORCE loss that we want to minimize for a single trajectory is:

$$\mathcal{L}(\theta) = -\sum_{t=0}^{T} G_t \log \pi_\theta(a_t \mid s_t).$$

2. Run the agent and plot its performance for different values of the learning rate.

Questions

a. Compared to the linear softmax policy, what has changed architecturally, and why does this make the policy more expressive?

b. Compare the performance of linear and MLP REINFORCE agents. Is there a significant improvement?

c. For which value of the learning rate do you find the best results?

4 Actor-Critic (week 7)

A key weakness of REINFORCE is high variance: the return G_t used to weight each gradient step is computed from a single Monte Carlo trajectory, which can vary greatly from episode to episode. This makes learning slow and unstable. The actor-critic architecture addresses this by replacing G_t with a lower-variance estimate based on a learned value function.

Architecture. The actor-critic maintains two separate networks:

- The **actor** $\pi_\theta(a \mid s)$: an MLP identical in structure to Section 3.2, parameterized by θ.
- The **critic** $V_\psi(s)$: an MLP that maps states to scalar value estimates, parameterized by ψ.

Advantage estimate. Instead of G_t, we weight each policy gradient step by the advantage:

$$\mathcal{A}_t = r_t + \gamma V_\psi(s_{t+1}) - V_\psi(s_t),$$

which is simply the one-step TD error of the critic. Intuitively, $\mathcal{A}_t > 0$ means the action taken was better than average (according to the critic), and the policy should be reinforced; $\mathcal{A}_t < 0$ means it was worse than average.

Critic and actor losses. After observing (s_t, a_t, r_t, s_{t+1}), the losses to minimize are:

$$\mathcal{L}_{critic}(\psi) = \frac{1}{2}\mathcal{A}_t^2 \qquad \text{(critic: minimize squared TD error)}$$

$$\mathcal{L}_{actor}(\theta) = -\mathcal{A}_t \log \pi_\theta(a_t|s_t) \qquad \text{(actor: policy gradient with advantage)}$$

Implement the following:

1. Complete the `ActorCriticAgent` class. Most of the PyTorch structure is provided; you need to fill in the advantage computation and `act` and `observe` functions.
2. Run the agent and plot its performance. Compare it with the REINFORCE MLP agent on the same plot.
3. Try different values for the actor's learning rate and the critic's learning rate.
4. Use `gif_episode` to visualize one episode of a well-trained actor-critic agent.

Questions

a. What are the key differences between REINFORCE and the actor-critic algorithm in terms of (i) when updates are performed, (ii) what signal is used to update the policy, and (iii) variance of the gradient estimate?

b. How does the actor-critic agent's learning curve compare to REINFORCE? Discuss differences in speed, stability, and final performance.

c. Which values for the actor's learning rate and the critic's learning rate give you the best performance?

d. Visualize one episode of the trained actor-critic agent. Describe the strategy it has learned. Does the behavior match your intuition about how a human would solve the task?